Servir modelos de IA entrenados con Docker

Comparativa de runtimes y servidores de inferencia

Material didáctico · IFCD107

20 de octubre de 2026

Resumen

Este documento explica las soluciones abiertas que pueden ejecutarse en contenedores Docker para obtener predicciones de modelos ya entrenados. Compara Ollama, LocalAI, vLLM, llama.cpp, NVIDIA Triton Inference Server y BentoML; describe qué aporta cada producto, qué artefactos necesita, cómo se integra mediante API y qué alternativa encaja con un modelo de regresión polinómica frente a un modelo generativo de lenguaje. La palabra *preentrenado* se usa con cuidado: normalmente el servidor aporta el motor y el usuario selecciona o monta los pesos del modelo.

Índice

1. Objetivo y alcance **2**

2. Entrenar, servir e inferir: tres etapas diferentes **2**
2.1. Qué significa que el modelo esté “entrenado” 2

3. Resumen comparativo **2**

4. Descripción detallada de las soluciones **3**
4.1. Ollama 3
4.2. LocalAI 3
4.3. vLLM 4
4.4. llama.cpp 4
4.5. NVIDIA Triton Inference Server 4
4.6. BentoML 4

5. Ejemplo práctico: modelo polinómico frente a Ollama **5**

6. Despliegue ilustrativo de Ollama con Compose **5**

7. Flujo de una predicción en un servidor general **6**

8. Criterios para elegir **6**

9. Recomendación para el caso IFCD107 **7**

10. Conclusiones **7**

11. Fuentes oficiales **7**

1. Objetivo y alcance

En este contexto, una solución de inferencia es un proceso que carga un modelo ya entrenado, recibe datos nuevos y devuelve una salida calculada. Docker empaqueta el servidor y sus dependencias para que pueda ejecutarse de forma repetible. No significa necesariamente que los pesos del modelo estén incluidos en la imagen: lo habitual es descargarlos al iniciar o montarlos desde un volumen.

La comparación se centra en software de código abierto y autoalojable. No existe una lista finita de todas las opciones, porque también se pueden construir servicios propios con bibliotecas de ML. Los ejemplos se centran en las seis alternativas destacadas en la consulta: Ollama, LocalAI, vLLM, llama.cpp, Triton y BentoML.

2. Entrenar, servir e inferir: tres etapas diferentes

1. **Entrenamiento:** el algoritmo ajusta sus parámetros usando datos de entrenamiento. Por ejemplo, scikit-learn ajusta un estimador de regresión polinómica.
2. **Empaquetado y despliegue:** se guarda el modelo y se prepara un servicio que pueda cargarlo, validar peticiones y exponer una API.
3. **Inferencia o predicción:** el servicio recibe observaciones nuevas y produce valores predichos. No vuelve a entrenar el modelo en cada petición.

Un servidor de inferencia suele gestionar carga del modelo, concurrencia, lotes, transporte HTTP o gRPC, métricas y ciclo de vida del proceso. El servidor no garantiza por sí solo que el modelo sea adecuado, preciso o actualizado: esas propiedades dependen del entrenamiento, de los datos y de la evaluación.

2.1. Qué significa que el modelo esté "entrenado"

Hay dos interpretaciones útiles:

- Un runtime como Ollama puede descargar modelos con pesos ya entrenados desde un catálogo compatible. El usuario decide qué modelo cargar; la imagen de Docker no implica que venga incluido un modelo concreto.
- Un servidor general como Triton espera que el usuario aporte sus propios artefactos y una configuración compatible. BentoML permite empaquetar modelo, dependencias y lógica de servicio en una unidad desplegable.

Además, que los pesos sean accesibles sin coste no convierte automáticamente el modelo en software libre: licencias, restricciones de uso y condiciones de redistribución cambian entre modelos. Conviene revisar la licencia del artefacto concreto.

3. Resumen comparativo

Solución	Uso principal	Modelo que se aporta	Cuándo elegirla
Ollama	Ejecutar LLMs localmente con una API sencilla.	Seleccionar y descargar un modelo del catálogo o usar un modelo compatible.	Pruebas locales, chat y generación con instalación simple.
LocalAI	Runtime de inferencia con API compatible con OpenAI y capacidades multimodales.	Elegir modelos y backends adecuados para cada modalidad.	Una interfaz común para texto, visión, audio, embeddings y otras tareas compatibles.
vLLM	Servir LLMs con enfoque en rendimiento y concurrencia.	Cargar un modelo compatible desde repositorio o disco.	API de generación para más carga o despliegue GPU.
llama.cpp	Inferencia eficiente de LLMs, especialmente modelos cuantizados GGUF.	Aportar un archivo de modelo compatible.	Ejecución local, CPU y equipos con recursos limitados; también admite aceleración según compilación y hardware.
NVIDIA Triton	Servir modelos de distintos frameworks y formatos mediante HTTP/gRPC.	Organizar uno o varios modelos en un repositorio de modelos.	Producción con varios modelos, lotes, métricas y hardware NVIDIA compatible.
BentoML	Construir y empaquetar un servicio de inferencia personalizado.	Aportar el modelo entrenado y definir la lógica de entrada/salida.	Modelos propios, como regresión, clasificación o pipelines Python.

La tabla describe el papel de cada servidor, no promete que cualquier modelo funcione en cualquier equipo. Compatibilidad de arquitectura, formato, precisión numérica, memoria, GPU y licencia deben comprobarse para el modelo elegido.

4. Descripción detallada de las soluciones

4.1. Ollama

Ollama ofrece una experiencia directa para ejecutar modelos de lenguaje y acceder a ellos mediante API. Su imagen oficial permite desplegar el proceso en Docker; un volumen persistente conserva los modelos descargados para que no haya que recuperarlos en cada recreación del contenedor. El servicio está orientado a cargar modelos y realizar inferencia generativa, no a servir cualquier estimador de scikit-learn.

Ventajas: puesta en marcha sencilla, catálogo accesible y API cómoda para prototipos. **Límites:** centrado en LLMs y sus tareas; el tamaño de los pesos y el contexto determinan memoria y velocidad. La aceleración depende del sistema anfitrión, los controladores y el soporte de Docker.

4.2. LocalAI

LocalAI es un runtime autoalojable que ofrece interfaces compatibles con APIs populares y selecciona backends según la modalidad/modelo. Su alcance incluye varias familias de tareas como

texto, visión, voz y embeddings. En una misma instancia se pueden configurar diferentes modelos, aunque la compatibilidad depende del backend y de la configuración.

Es útil cuando se desea un endpoint parecido al de un proveedor de API y se quiere alojar el cómputo en infraestructura propia. No debe asumirse que todo modelo o formato queda disponible automáticamente: hay que configurar el modelo, backend y parámetros.

4.3. vLLM

vLLM es un motor de inferencia y servicio para modelos de lenguaje, con servidor de API compatible con OpenAI. Se suele elegir para cargas con más peticiones o necesidad de servir modelos grandes en GPU. El modelo se proporciona por identificador o ruta local y se descargan o montan los pesos según el despliegue.

La compatibilidad concreta depende de arquitectura, versión del runtime, acelerador, memoria y configuración. Para una API sencilla de chat en un portátil puede ser más trabajo operativo del necesario; para producción GPU puede ser una opción apropiada.

4.4. llama.cpp

llama.cpp es un motor de inferencia popular para modelos de lenguaje, en especial los distribuidos en formato GGUF y con cuantización. La cuantización reduce el tamaño y los requisitos de memoria a costa de posibles cambios de calidad. Puede servir un modelo local a través de un servidor HTTP; se debe elegir un archivo compatible y configurar el hardware/aceleración que se tenga.

Se adapta bien a escenarios locales y equipos modestos, pero la experiencia y el rendimiento varían con el modelo, versión, parámetros, CPU/GPU y construcción utilizada.

4.5. NVIDIA Triton Inference Server

Triton es un servidor de inferencia general: sirve modelos preparados en distintos formatos/frameworks, como ONNX Runtime, TensorRT, PyTorch y TensorFlow, sujeto a los backends disponibles en la versión instalada. El servidor recibe uno o más repositorios de modelos y publica interfaces HTTP y gRPC. Está pensado para operación y rendimiento, y requiere configurar cuidadosamente el artefacto, la firma de entrada/salida y el backend.

Es más general que un runtime de chat, pero normalmente requiere más preparación. Aunque puede ejecutarse sin GPU para los modelos compatibles con CPU, el uso de aceleración depende de contenedor, GPU NVIDIA, controladores y runtime de contenedores.

4.6. BentoML

BentoML es un marco de empaquetado y servicio. Se escribe una definición del servicio que carga el artefacto entrenado, valida entradas y expone operaciones. Después se construye un paquete y se genera una imagen Docker ejecutable. A diferencia de Ollama, el foco es permitir que el desarrollador lleve su propio modelo y defina la API.

Esto lo hace especialmente adecuado para publicar predicciones de un modelo scikit-learn: se conserva el entrenamiento y se añade una capa HTTP con validación, serialización y manejo de errores. La responsabilidad de garantizar que el modelo guardado se cargue de forma segura y compatible recae en el proyecto.

5. Ejemplo práctico: modelo polinómico frente a Ollama

Supongamos que el proyecto IFCD107 ajusta un modelo de regresión polinómica con datos numéricos. El artefacto contiene la transformación polinómica y el estimador entrenado. Para predecir una nueva observación, la API recibe el valor de entrada, lo transforma con el mismo pipeline y devuelve un número.

Listing 1: Entrenamiento ilustrativo de una canalización polinómica

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

modelo = make_pipeline(
    PolynomialFeatures(degree=2, include_bias=False),
    LinearRegression()
)
modelo.fit(X_entrenamiento, y_entrenamiento)
```

Después se guarda el pipeline completo, no únicamente el último estimador. El servicio carga ese artefacto al arrancar y llama a `predict` con las entradas válidas. Una petición puede ser JSON con una lista de variables y la respuesta JSON con el valor estimado. Así se evita repetir entrenamiento en cada llamada.

Ollama tiene otra función: recibe texto de conversación y devuelve texto generado por un LLM. Puede ayudar a redactar una explicación en lenguaje natural sobre el resultado, pero no debe sustituir al modelo numérico ni inventar el valor de regresión. Una arquitectura posible es:

1. La API numérica calcula la predicción con el pipeline polinómico.
2. Opcionalmente, la aplicación envía el valor calculado y contexto acotado al modelo de lenguaje para generar una explicación.
3. La interfaz presenta la predicción numérica como dato del modelo y la explicación como texto generado.

La predicción verificable debe provenir del modelo de regresión; el LLM solo redacta texto a partir de ese resultado.

6. Despliegue ilustrativo de Ollama con Compose

El ejemplo inicia el runtime y conserva los pesos descargados en un volumen con nombre. La descarga de un modelo concreto se realiza después de iniciar el servicio.

Listing 2: compose.yaml mínimo para el servicio de Ollama

```
services:
  ollama:
    image: ollama/ollama
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama

volumes:
  ollama_data:
```

Una vez levantado el servicio, se puede descargar/ejecutar un modelo con la CLI del contenedor. La API de Ollama escucha en el puerto expuesto. Para GPU hay que seguir las instrucciones vigentes de la imagen y del hardware anfitrión; no basta con añadir un parámetro de GPU si los controladores/runtime no están instalados.

7. Flujo de una predicción en un servidor general

1. El artefacto del modelo se coloca en el formato y estructura requeridos por el servidor.
2. Se configura el nombre, versión, backend y esquema de entradas/salidas.
3. Docker inicia el servidor y monta el artefacto o la configuración persistente.
4. El cliente envía características por HTTP o gRPC.
5. El servidor valida, ejecuta inferencia y devuelve resultados y, en su caso, métricas.

En un producto educativo se recomienda empezar por una API pequeña con un endpoint claro, documentar el esquema JSON y manejar errores de entrada. El modelo no debe aceptar columnas en orden ambiguo ni datos sin la misma transformación usada durante el entrenamiento.

8. Criterios para elegir

Tipo de modelo
Para chat/LLM, Ollama, LocalAI, vLLM o llama.cpp; para un estimador tabular propio, BentoML o Triton.

Facilidad de uso
Ollama suele reducir el trabajo de arranque para LLM local; BentoML facilita convertir código Python propio en servicio.

Escala y concurrencia
vLLM se centra en servir LLMs; Triton cubre múltiples backends de inferencia y escenarios de servicio más generales.

Hardware
Confirmar soporte de CPU/GPU, memoria de vídeo, arquitectura de GPU, controladores y runtime de Docker antes de elegir.

Formato del modelo
Comprobar formato admitido y versiones compatibles; un modelo entrenado en un framework puede requerir exportación o conversión.

Operación
Valorar volúmenes para persistencia, salud del contenedor, logs, métricas, límites de recursos, seguridad de API y estrategia de actualización.

Licencia
Revisar tanto la licencia del servidor como la del modelo/pesos concretos y los permisos de uso comercial o redistribución.

9. Recomendación para el caso IFCD107

Para el modelo de regresión polinómica, una solución proporcionada es mantener el pipeline de scikit-learn y encapsularlo en un endpoint de predicción con BentoML (o una API Flask/FastAPI si el objetivo didáctico es construir el servicio manualmente). Ollama puede permanecer como servicio adicional para generar explicaciones en lenguaje natural, claramente separado del cálculo numérico.

Triton tiene sentido cuando el ejercicio introduce despliegue de varios modelos, formatos interoperables, gRPC, batching o aceleración de producción. vLLM y llama.cpp son opciones de lenguaje generativo; LocalAI proporciona una capa multipropósito para varios tipos de inferencia. No son intercambiables sin más con el endpoint del modelo polinómico.

10. Conclusiones

- Docker distribuye y ejecuta el servicio; el modelo puede estar en un volumen o descargarse por separado.
- Ollama, LocalAI, vLLM y llama.cpp se orientan principalmente a runtimes de modelos generativos, en especial LLMs.
- Triton y BentoML sirven modelos aportados por el equipo, aunque con distinto nivel de abstracción y configuración.
- Un sistema puede combinar un modelo numérico determinista con un LLM para explicar resultados, manteniendo separadas predicción y generación de texto.
- La elección final depende del modelo, formato, hardware, carga, licencias y esfuerzo operativo.

11. Fuentes oficiales

Las páginas consultadas describen instalación, APIs, empaquetado y compatibilidad; sus detalles pueden cambiar entre versiones. Se recomienda revisar la documentación de la versión seleccionada antes del despliegue.

1. Ollama, imagen Docker oficial: https://ollama.com/blog/ollama-is-now-available-as-an-official-d
2. LocalAI, documentación y ejecución Docker: https://localai.io/docs/index.html
3. vLLM, documentación: https://docs.vllm.ai/
4. llama.cpp, repositorio oficial: https://github.com/ggml-org/llama.cpp
5. NVIDIA Triton Inference Server, guía de usuario: https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/
6. BentoML, empaquetado para despliegue: https://docs.bentoml.org/en/latest/get-started/packaging-for-deployment.html
7. Hugging Face Text Generation Inference (TGI), estado del proyecto: https://github.com/huggingface/text-generation-inference